Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

LINUX LOG MONITORING & ALERT SYSTEM

Submitted by

VAMSI (2411021060894)

YATEESH KUMAR REDDY (2411021060654)

SHASHANK (2411021060397)

SANTHOSH (2411021060891)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Mrs. Kusuma J

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Linux Log Monitoring & Alert System"** submitted by Vamsi [2411021060894], Yateesh Kumar Reddy [2411021060654], Shashank [2411021060397] and Santhosh [2411021060891] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work during the academic year 2026-2027.

Mrs. Kusuma J	Prof. Ananthanagu U	Dr. Shekhar R
Course Handling Faculty	Course Coordinator	Professor & HoD

Table of Contents

1. Introduction
- 1.1 Aim
- 1.2 Background
- 1.3 Problem Statement
- 1.4 Objectives
- 1.5 Scope of the Project

2. Methodology
- 2.1 Tools and Technologies Used
- 2.2 Approach / Development Process
- 2.3 Functional Requirements
- 2.4 System Workflow

3. Linux System Administration – Design and Implementation
- 3.1 System Architecture
- 3.2 Linux Environment Setup
- 3.3 Project Directory Structure and Sample Data
- 3.4 Bash Monitoring Engine
- 3.5 Log Searching and Analysis
- 3.6 Failed-Login Detection
- 3.7 Alert Generation and Storage
- 3.8 Report Generation
- 3.9 Viewing Linux System Logs
- 3.10 Python Backend and API
- 3.11 Web Dashboard
- 3.12 Security Considerations and Limitations

4. Results and Testing
- 4.1 Implementation Results
- 4.2 Test Environment
- 4.3 Testing and Validation
- 4.4 Testing Limitations

5. Conclusion and Future Scope
- 5.1 Conclusion
- 5.2 Achievements
- 5.3 Current Limitations
- 5.4 Future Scope

References

Appendices

1. Introduction

1.1 Aim

The aim of this project is to design, develop, and demonstrate a Linux Log Monitoring & Alert System that runs locally on an Ubuntu Linux machine. The system reads log files, searches them for events of interest, raises alerts when important keywords appear, checks for failed-login entries, analyses the log contents, and produces a text summary report.

Bash shell scripts form the core monitoring and log-processing engine. A lightweight Python 3 server connects those scripts to a web dashboard built with HTML, CSS, and JavaScript, so that the same operations can be used from a browser as well as from the terminal.

1.2 Background

Linux systems record their activity in log files and in the systemd journal. Authentication attempts, service start-ups, kernel messages, and application errors are all written to logs, which administrators examine to diagnose faults and to detect suspicious behaviour. On Ubuntu, many of these files are stored under `/var/log`, and recent journal entries can be read with the `journalctl` command.

Reading logs manually becomes slow and error-prone as they grow, and important lines such as errors or failed logins are easily missed. Simple automation built from standard Linux tools can filter, count, and highlight such events. This project applies that idea using Bash scripting and a small local web interface, developed and demonstrated in an Ubuntu Linux environment running in VirtualBox.

1.3 Problem Statement

Administrators who inspect log files by hand must scroll through many lines to find errors, critical events, and failed login attempts. Without an automated mechanism, new events are noticed late, searches are repeated manually, and no record of the alerts raised is kept.

Therefore, this project develops a local log monitoring and alert system that watches a log file, generates keyword-based alerts, stores those alerts, supports searching and analysis, checks for failed-login entries, generates a summary report, and offers a dashboard from which these functions can be operated.

1.4 Objectives

The main objectives of the Linux Log Monitoring & Alert System are:

- To monitor a log file automatically and raise an alert when an error, critical, or failed-login entry is appended.

- To store generated alerts in files so that an alert history can be reviewed later.
- To search log entries by a keyword supplied by the user.
- To detect failed-login entries and save an alert file for them.
- To analyse a log and report the number of total, error, critical, and failed-login entries.
- To generate a readable text summary report of the log.
- To view selected Linux system logs in a read-only manner, where file permissions allow.
- To provide a local web dashboard that exposes these functions in a browser.

1.5 Scope of the Project

The project operates on a single Ubuntu Linux machine. Its monitoring is directed mainly at a sample log file (`logs/sample.log`), and it can also display selected Linux system logs where the file permissions allow. The web dashboard runs locally and is reached through the loopback address `http://127.0.0.1:8000`.

The project does not use a database, cloud services, machine learning, remote monitoring, or email notifications, and no Cron-based scheduling is part of the implementation. Monitoring is started and stopped by the user. These items are listed as possible future work in Section 5.4.

2. Methodology

2.1 Tools and Technologies Used

2.1.1 Software Requirements

The system was developed and demonstrated on Ubuntu Linux using Bash scripting, Python 3, and standard web technologies. Table 2.1 lists the tools and their roles.

Table 2.1: Tools and technologies used

Tool / Technology	Role in the project
Ubuntu Linux 26.04.1 LTS	Operating system used for development and demonstration (Figure 3.2).
VirtualBox	Virtualisation software used to run Ubuntu on the development computer.
Bash shell scripts	Core monitoring and log-processing engine: monitoring, searching, analysis, failed-login detection, report generation, and log viewing.
Python 3	Lightweight local HTTP server and API bridge between the web dashboard and the Bash scripts (`server.py`).
HTML, CSS, JavaScript	Web dashboard (`web/index.html`) showing status, counters, log entries, system logs, and alerts.
nano text editor	Used in the terminal to write and edit the scripts (visible in Figures 3.8 to 3.10).
Linux utilities	Commands such as `mkdir`, `ls`, `cat`, `chmod`, `find`, `whoami`, `hostname`, `lsb_release`, and `journalctl` were used during setup and testing.
Text files	`logs/sample.log`, `alerts/automatic_alerts.txt`, `alerts/failed_login_alerts.txt`, and `reports/log_summary.txt` store log data, alerts, and reports.

2.1.2 Hardware and Environment

The project was implemented on a computer running Ubuntu Linux inside a VirtualBox virtual machine. The general hardware requirements are:

- A laptop or desktop computer.
- A compatible processor capable of running Ubuntu Linux.
- Sufficient RAM to run the operating system and the required utilities.
- Adequate storage space for scripts, logs, alerts, and reports.

2.2 Approach / Development Process

The project was developed in stages, from environment preparation to testing and documentation. The screenshots collected during the work show terminal activity on 14 October 2026 and the web dashboard on 15 October 2026.

Step 1: Problem Identification. The need for automatic detection of errors, critical events, and failed logins in Linux logs was identified, and the objectives in Section 1.4 were defined.

Step 2: Requirement Analysis. The required functions were listed (Section 2.3), and Bash, Python 3, and a browser-based dashboard were chosen to implement them.

Step 3: Environment Setup. Ubuntu Linux was prepared, and the project directory with the folders `scripts`, `logs`, `reports`, and `alerts` was created from the terminal.

Step 4: Sample Data Creation. A sample log containing INFO, ERROR, FAILED, and CRITICAL entries was created so that every function could be tested with known data.

Step 5: Bash Script Development. Separate scripts were written and made executable for analysis, keyword search, failed-login detection, report generation, real log viewing, and automatic monitoring. An interactive terminal menu was built to call them, and it was extended from eight to ten options.

Step 6: Automatic Monitoring. The automatic monitor was tested by appending new entries to the sample log and observing the alerts produced.

Step 7: Backend and Dashboard. A Python 3 local server and an HTML, CSS, and JavaScript dashboard were created so that the Bash functions could be used from a browser.

Step 8: Testing. Each function was run and its output compared with the contents of the sample log (Chapter 4).

Step 9: Documentation. Screenshots were collected as evidence, diagrams were prepared, and this report was written.

2.3 Functional Requirements

Table 2.2 lists the functional requirements, the component that provides each one, and where its evidence appears in this report.

Table 2.2: Functional requirements

ID	Requirement	Component	Evidence
FR1	View log entries from the sample log	Menu / dashboard	Figures 3.6, 3.16
FR2	Search the log by a keyword	search_logs.sh / dashboard	Figure 3.9 (terminal)
FR3	Detect failed-login entries and save an alert file	failed_login.sh	Figure 3.10
FR4	Analyse the log and count event types	analyzer.sh	Figure 3.8
FR5	Generate and save a text summary report	generate_report.sh	Figures 3.11, 3.12
FR6	Raise and store keyword-based alerts automatically	auto_monitor.sh	Figures 4.2, 4.3, 4.4

ID	Requirement	Component	Evidence
FR7	Show alert history	Menu / dashboard	Figure 3.18
FR8	View selected Linux system logs (read-only)	real_log_viewer.sh / dashboard	Figures 3.13, 3.14, 3.17
FR9	Start and stop monitoring and refresh the display	Dashboard controls	Controls visible in Figure 3.15
FR10	Show status and counters on a dashboard	Terminal and web dashboards	Figures 3.15, 4.5

2.4 System Workflow

The central workflow of the system is automatic monitoring. When monitoring is started, the monitor script watches the sample log. If a new line contains an alert keyword (ERROR, CRITICAL, or FAILED, as observed in the tests), an alert line with a timestamp is displayed and saved to the alert file. The administrator can then review the alerts from the terminal menu or from the alert history panel of the dashboard. Searching, analysis, failed-login detection, and report generation can be run at any time. Figure 2.1 illustrates this workflow.

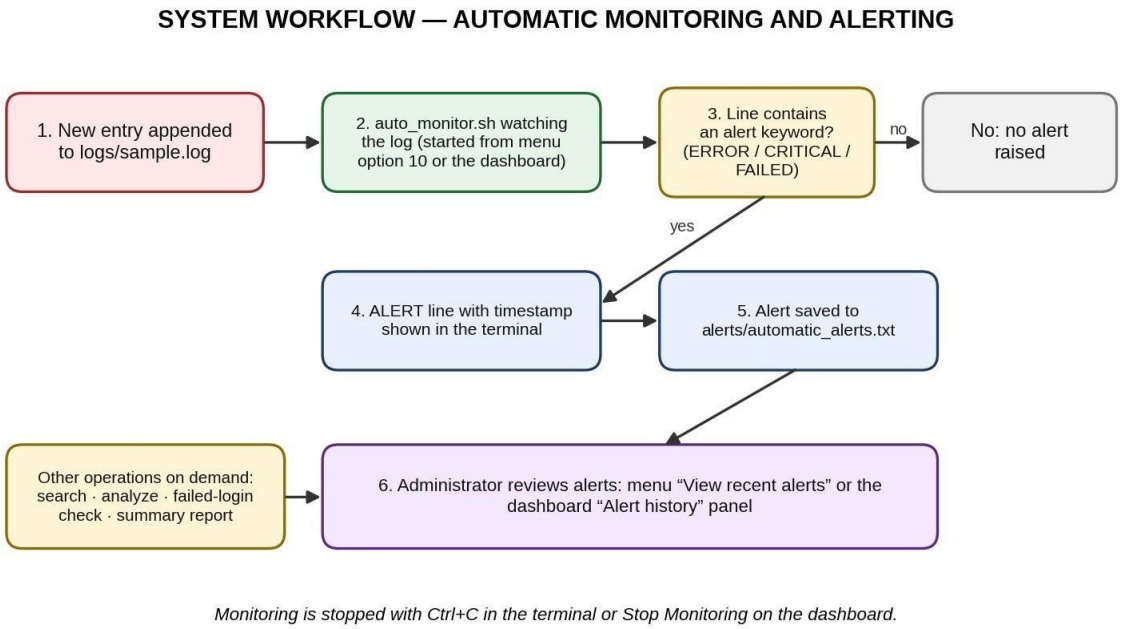


Figure 2.1: System workflow of the Linux Log Monitoring & Alert System

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The Linux Log Monitoring & Alert System is a local, layered application running inside an Ubuntu Linux virtual machine. The user works either through an interactive terminal menu or through a web dashboard in a browser. Bash scripts perform the monitoring and log processing. A Python 3 server connects the dashboard to those scripts. Log files are read from the project and from the Linux system, and alerts and reports are written to text files.

Figure 3.1 presents the system architecture and the relationships between its components.

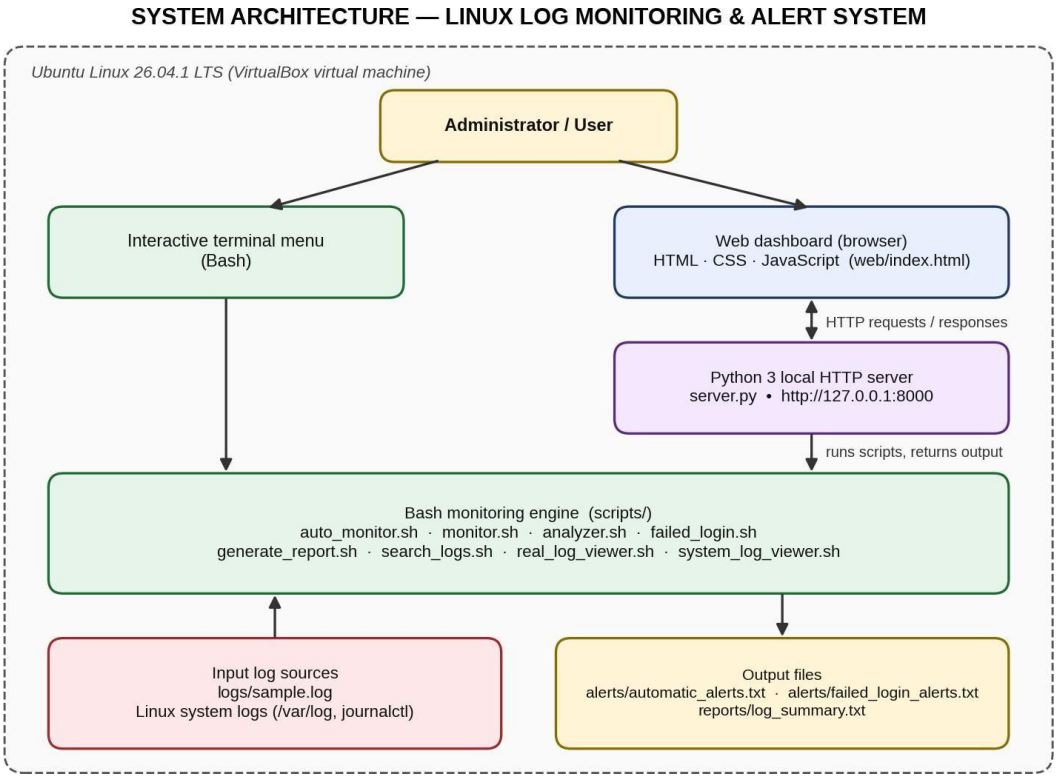


Figure 3.1: System architecture of the Linux Log Monitoring & Alert System

Components and their roles

- **Administrator / User:** operates the system from the terminal menu or the web dashboard.
- **Interactive terminal menu (Bash):** lets the user run the individual functions by choosing a number.
- **Web dashboard (HTML, CSS, JavaScript):** shows monitoring status, counters, recent log entries, system logs, and alert history, and offers buttons for the operations.
- **Python 3 local HTTP server:** serves the dashboard at `http://127.0.0.1:8000` and passes dashboard requests to the Bash scripts.

- **Bash monitoring engine:** the scripts in the `scripts/` directory that perform monitoring, searching, analysis, failed-login detection, report generation, and log viewing.
- **Input log sources:** `logs/sample.log` and selected Linux system logs, such as those under `/var/log` and the systemd journal.
- **Output files:** alert files in `alerts/` and the summary report in `reports/`.

3.2 Linux Environment Setup

3.2.1 Linux Distribution and Version

The project was developed on Ubuntu Linux. The user account, host name, and distribution details were checked with the commands `whoami`, `hostname`, and `lsb_release -a`. Figure 3.2 shows that the account is yateesh, the host name is Ubuntu, and the system is Ubuntu 26.04.1 LTS (release 26.04, codename resolute).

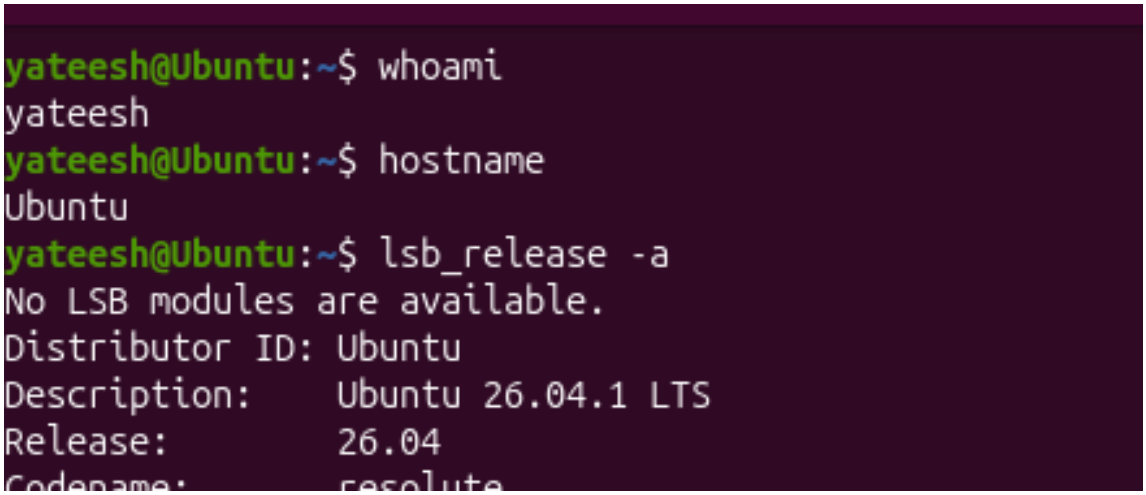


Figure 3.2: Ubuntu version, host name, and user details

3.3 Project Directory Structure and Sample Data

3.3.1 Directory Creation

The project directory `linux-log-monitor` was created in the user's home directory with a single `mkdir -p` command that also created the subdirectories `scripts`, `logs`, `reports`, and `alerts`. In Figure 3.3, the `pwd` command confirms the location `/home/yateesh/linux-log-monitor`, and `ls -R` lists the four newly created, empty subdirectories.



Figure 3.3: Creation of the project directory and its subdirectories

3.3.2 Project Files

After the scripts and data files were created, the command `find . -maxdepth 3 -type f | sort` listed all files in the project. Figure 3.4 shows eleven files: two alert files, the sample log, the summary report, and seven scripts. Table 3.1 explains the purpose of each item.

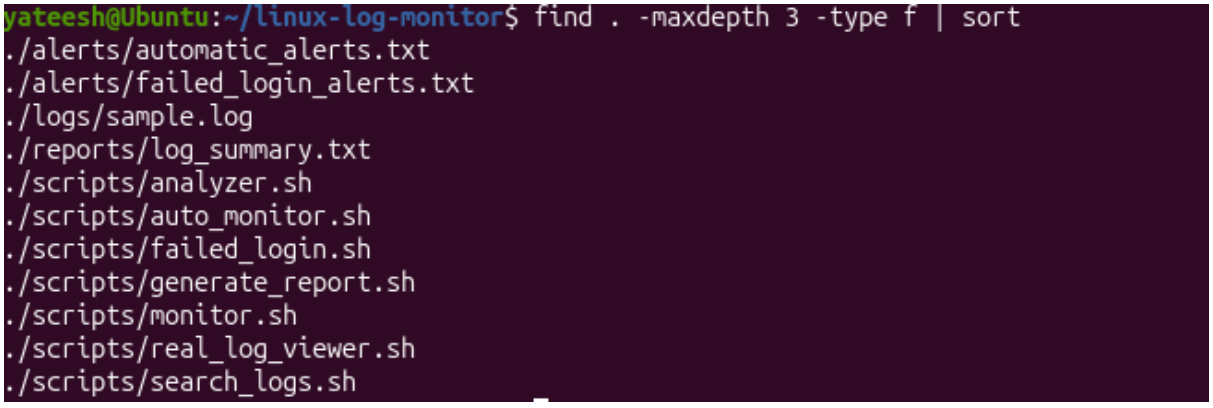


Figure 3.4: Project files listed with the find command

Table 3.1: Project files and their purpose

File	Purpose
scripts/monitor.sh	Monitoring script of the project (listed in Figure 3.4).
scripts/auto_monitor.sh	Watches the sample log and raises alerts for new matching entries.
scripts/analyzer.sh	Counts total, error, critical, and failed-login entries and prints the log.
scripts/search_logs.sh	Prompts for a keyword and lists matching log lines with line numbers.
scripts/failed_login.sh	Lists failed-login entries and saves an alert file.
scripts/generate_report.sh	Creates the log summary report and saves it to the reports directory.

File	Purpose
scripts/real_log_viewer.sh	Menu for viewing the authentication log, system log, system journal, or sample log.
scripts/system_log_viewer.sh	Viewer for selected Linux system logs (not shown in the screenshots).
logs/sample.log	Sample log used for monitoring, searching, and analysis.
alerts/automatic_alerts.txt	Alerts raised by the automatic monitor.
alerts/failed_login_alerts.txt	Alerts saved by the failed-login detector.
reports/log_summary.txt	Summary report generated from the sample log.
server.py, web/index.html	Python server and web dashboard (not shown in Figure 3.4).

3.3.3 Sample Log

A sample log was written to `logs/sample.log` with a `cat > ... <<'EOF'` command so that every function could be tested against known content. Each line has the form date, time, level, and message. As Figure 3.5 shows, the file contains eight entries: INFO, ERROR, FAILED, and CRITICAL levels, with two ERROR lines, one CRITICAL line, and two FAILED lines.

```
yateesh@Ubuntu:~/linux-log-monitor$ cat > logs/sample.log <<'EOF'
2026-10-14 09:00:00 INFO System started
2026-10-14 09:05:12 ERROR Disk usage warning
2026-10-14 09:10:20 FAILED Failed login for user admin
2026-10-14 09:11:05 FAILED Failed login for user guest
2026-10-14 09:12:30 INFO User yateesh logged in
2026-10-14 09:15:45 CRITICAL Service unavailable
2026-10-14 09:20:00 ERROR Application connection failed
2026-10-14 09:25:00 INFO Monitoring initialized
EOF
yateesh@Ubuntu:~/linux-log-monitor$ cat logs/sample.log
2026-10-14 09:00:00 INFO System started
2026-10-14 09:05:12 ERROR Disk usage warning
2026-10-14 09:10:20 FAILED Failed login for user admin
2026-10-14 09:11:05 FAILED Failed login for user guest
2026-10-14 09:12:30 INFO User yateesh logged in
2026-10-14 09:15:45 CRITICAL Service unavailable
2026-10-14 09:20:00 ERROR Application connection failed
2026-10-14 09:25:00 INFO Monitoring initialized
```

Figure 3.5: Creation and display of the sample log

3.4 Bash Monitoring Engine

The monitoring engine is a set of Bash scripts. Each script handles one task and was made executable with `chmod +x` before use. This modular arrangement allows each function to be tested separately and to be called both from the terminal menu and from the Python server.

3.4.1 Interactive Terminal Menu

An interactive menu brings the scripts together. The first version, titled LINUX LOG MONITORING & ALERT SYSTEM, offered eight options: view log entries, search by keyword, detect failed logins, generate a summary report, view recent alerts, view system information, run full log analysis, and exit. In Figure 3.6, option 1 displays the eight sample log entries and waits for Enter before returning to the menu.

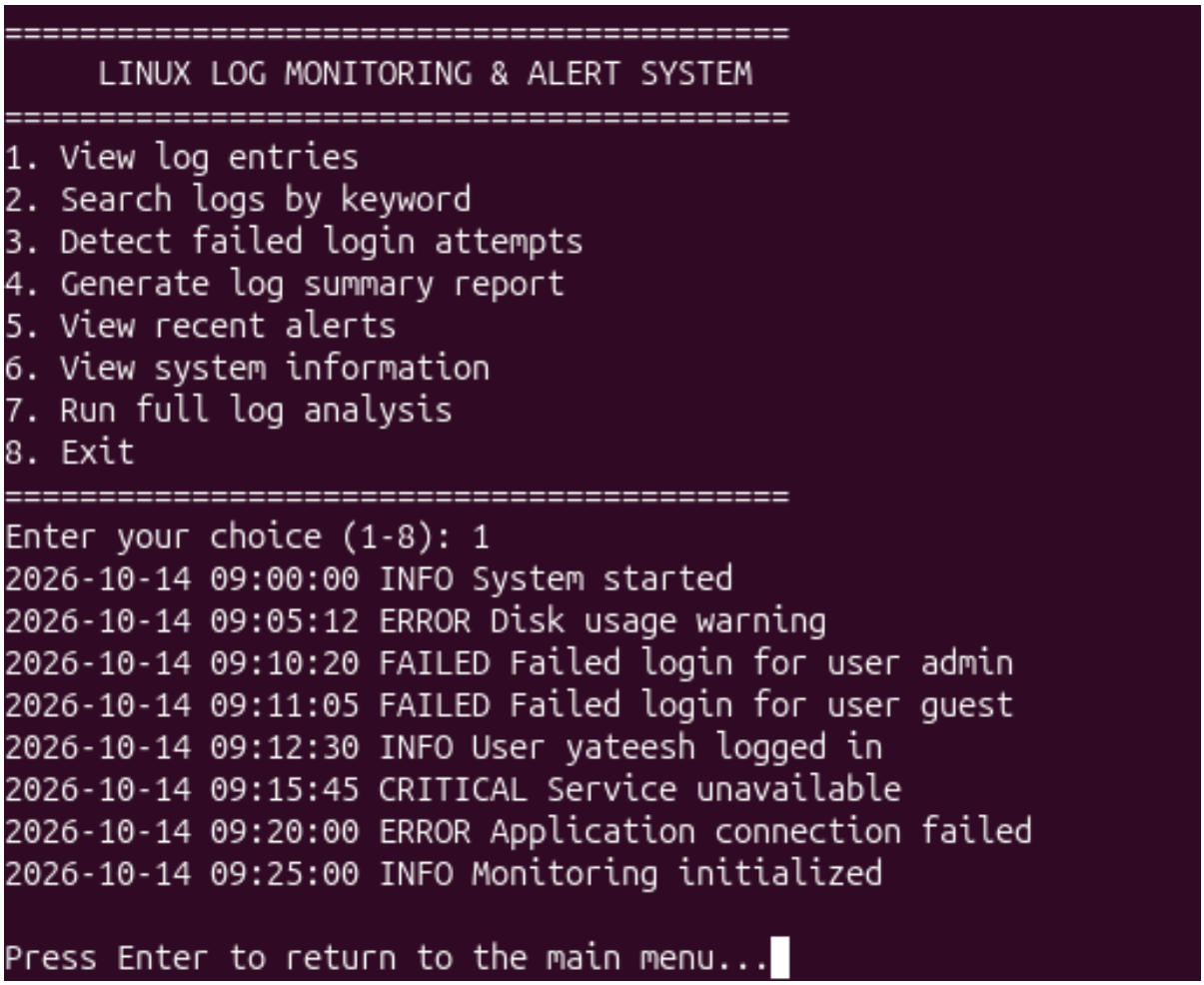


Figure 3.6: Terminal menu showing the log entries (option 1)

The menu checks its input. Figure 3.7 shows that entering 99 produces the message “Invalid choice. Please enter a number from 1 to 8.” and returns the user to the menu.

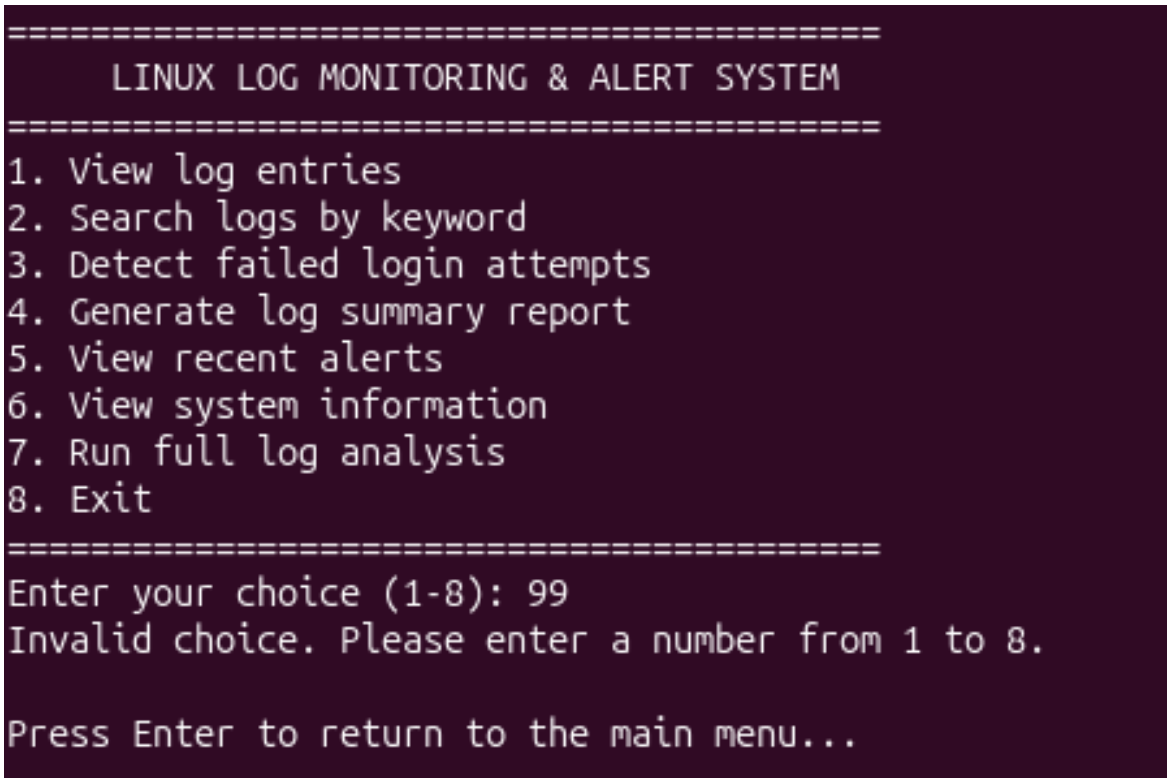


Figure 3.7: Menu response to an invalid choice

Later screenshots (Figures 4.4 and 4.5) show an extended ten-option version titled LINUX LOG MONITORING SYSTEM, which adds the options to view real Linux logs and to start automatic log monitoring.

3.4.2 Automatic Log Monitor

The script `auto_monitor.sh` monitors `logs/sample.log`. It prints the monitored path and the message "Press Ctrl+C to stop.", and then displays an ALERT line, prefixed with a timestamp, for each new log entry that contains an alert keyword. The alerts are also stored in `alerts/automatic_alerts.txt`. The tests of this script are presented in Section 4.1 (Figures 4.1 to 4.4).

3.5 Log Searching and Analysis

3.5.1 Log Analysis

The script `analyzer.sh` counts the entries in the log by type and prints the complete log. Figure 3.8 shows its output for the sample log: 8 total entries, 2 error entries, 1 critical entry, and 2 failed-login entries. These counts agree with the contents of the sample log shown in Figure 3.5.

```
yateesh@Ubuntu:~/linux-log-monitor$ nano scripts/analyzer.sh
yateesh@Ubuntu:~/linux-log-monitor$ chmod +x scripts/analyzer.sh
./scripts/analyzer.sh
================================
      LINUX LOG ANALYZER
================================

Total log entries:
8

Error entries:
2

Critical entries:
1

Failed login entries:
2

Complete log entries:
2026-10-14 09:00:00 INFO System started
2026-10-14 09:05:12 ERROR Disk usage warning
2026-10-14 09:10:20 FAILED Failed login for user admin
2026-10-14 09:11:05 FAILED Failed login for user guest
2026-10-14 09:12:30 INFO User yateesh logged in
2026-10-14 09:15:45 CRITICAL Service unavailable
2026-10-14 09:20:00 ERROR Application connection failed
2026-10-14 09:25:00 INFO Monitoring initialized
```

Figure 3.8: Output of the log analyzer script

3.5.2 Keyword Search

The script `search_logs.sh` asks for a keyword and prints the matching lines, each preceded by its line number. In Figure 3.9 the script was run twice. Searching for ERROR returned lines 2 and 7. Searching for FAILED returned lines 3, 4, and 7. Line 7 is “ERROR Application connection failed”; it matched because the search is not case-sensitive and the word “failed” appears in the message. This shows that the search matches the keyword anywhere in a line, not only at the log-level field.

```
yateesh@Ubuntu:~/linux-log-monitor$ nano scripts/search_logs.sh
yateesh@Ubuntu:~/linux-log-monitor$ chmod +x scripts/search_logs.sh
./scripts/search_logs.sh
================================
        LOG SEARCH
================================
Enter keyword to search: ERROR

Search results for: ERROR
--------------------------------
2:2026-10-14 09:05:12 ERROR Disk usage warning
7:2026-10-14 09:20:00 ERROR Application connection failed
yateesh@Ubuntu:~/linux-log-monitor$ chmod +x scripts/search_logs.sh
./scripts/search_logs.sh
================================
        LOG SEARCH
================================
Enter keyword to search: FAILED

Search results for: FAILED
--------------------------------
3:2026-10-14 09:10:20 FAILED Failed login for user admin
4:2026-10-14 09:11:05 FAILED Failed login for user guest
7:2026-10-14 09:20:00 ERROR Application connection failed
```

Figure 3.9: Keyword search for ERROR and FAILED

3.6 Failed-Login Detection

The script `failed_login.sh` displays the heading FAILED LOGIN DETECTOR, lists the failed-login entries found in the log, and saves an alert file. Figure 3.10 shows the two entries detected (lines 3 and 4, for the users admin and guest) and the message that the alert was saved in the `alerts` directory of the project. The file `failed_login_alerts.txt` also appears in the project file listing in Figure 3.4.

The detection in this project is performed on the sample log. It identifies lines labelled FAILED and is not a substitute for analysing the system's real authentication records.

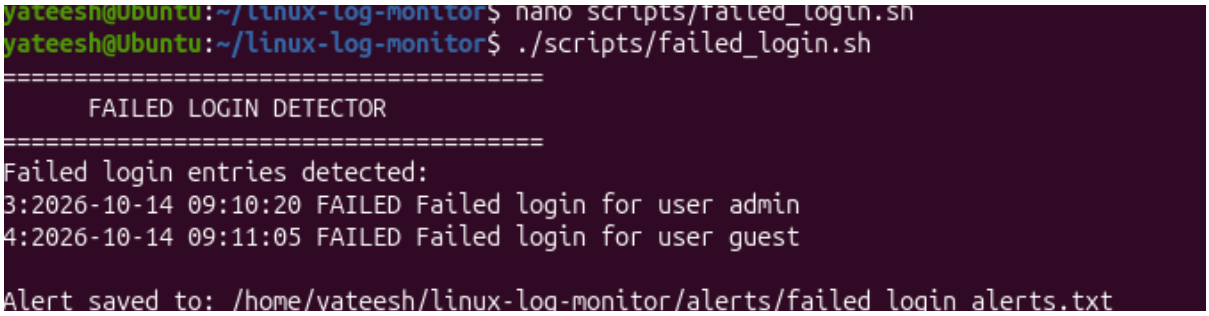


Figure 3.10: Failed-login detection and alert file creation

3.7 Alert Generation and Storage

Alerts are generated in two ways. The failed-login script saves its findings to `alerts/failed_login_alerts.txt`. The automatic monitor writes each alert, in the form "[date time] ALERT: <log line>", to `alerts/automatic_alerts.txt`. The terminal menu provides an option to view recent alerts, and the dashboard shows the stored alerts in its Alert history panel (Figure 3.18). Because alerts are kept in plain text files, they can be read with ordinary Linux commands such as `cat`, as demonstrated in Figure 4.3.

3.8 Report Generation

The script `generate_report.sh` produces a log summary report. Figure 3.11 shows the report on the terminal. It carries the title LINUX LOG MONITORING SYSTEM and Log Summary Report, the generation time, the source file, the counts (8 entries, 2 errors, 1 critical event, 2 failed-login attempts), and the lists of errors, critical events, and failed-login attempts with their line numbers. The final line states that the report was saved to `reports/log_summary.txt`.

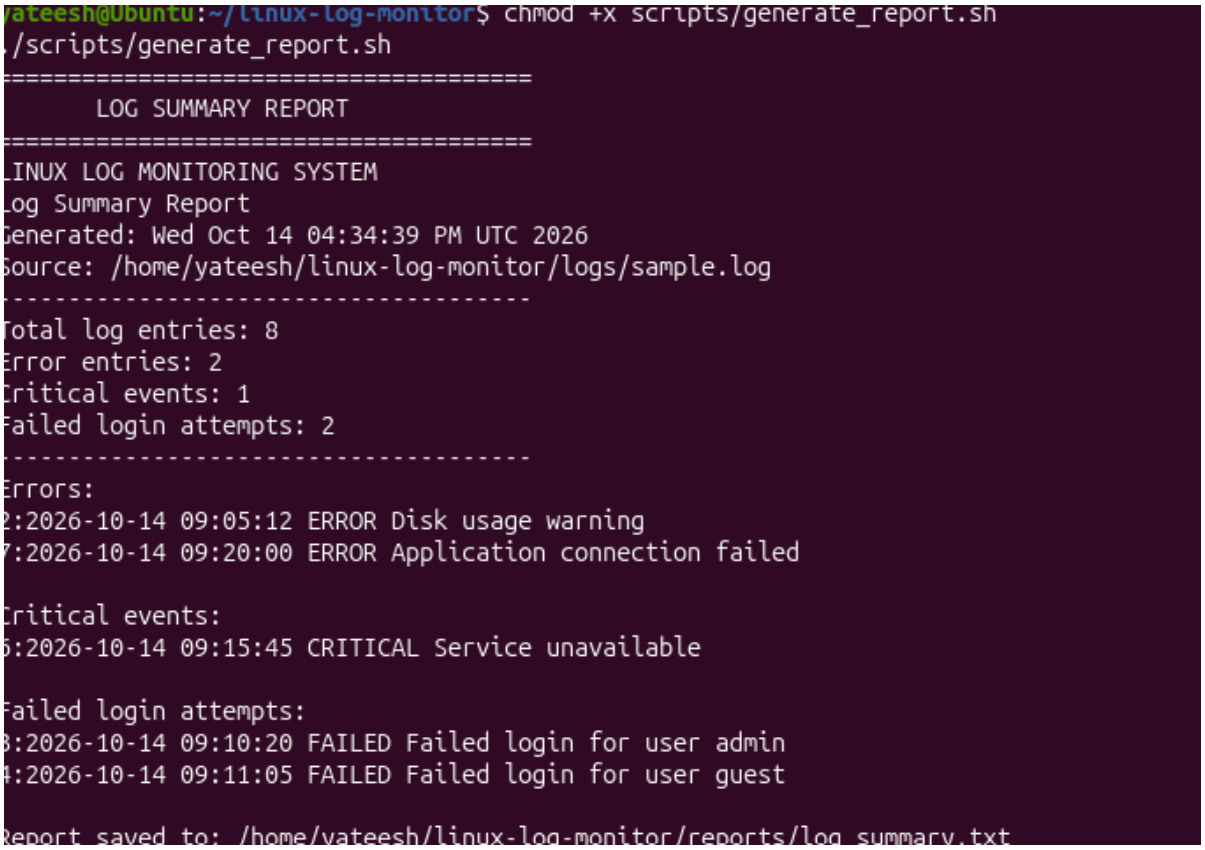


Figure 3.11: Log summary report generated on the terminal

The saved report was then displayed with `cat reports/log_summary.txt`. Figure 3.12 confirms that the stored file contains the same content as the report shown on the screen.

```
yateesh@Ubuntu:~/linux-log-monitor$ cat reports/log_summary.txt
LINUX LOG MONITORING SYSTEM
Log Summary Report
Generated: Wed Oct 14 04:34:39 PM UTC 2026
Source: /home/yateesh/linux-log-monitor/logs/sample.log
------------------------------------
Total log entries: 8
Error entries: 2
Critical events: 1
Failed login attempts: 2
------------------------------------
Errors:
2:2026-10-14 09:05:12 ERROR Disk usage warning
7:2026-10-14 09:20:00 ERROR Application connection failed

Critical events:
6:2026-10-14 09:15:45 CRITICAL Service unavailable

Failed login attempts:
3:2026-10-14 09:10:20 FAILED Failed login for user admin
4:2026-10-14 09:11:05 FAILED Failed login for user guest
```

Figure 3.12: Contents of the saved report file

3.9 Viewing Linux System Logs

In addition to the sample log, the project can display selected Linux system logs. System log files are normally protected by file permissions. In Figure 3.13, `ls -l /var/log/auth.log` and `ls -l /var/log/syslog` show the permissions `-rw-r` with owner syslog and group adm, which means that only the owner and members of the adm group can read these files. The `journalctl -n 10 --no-pager` command prints the ten most recent journal entries, which in this screenshot are service messages from systemd (for example, anacron, systemd-tmpfiles-clean, and sysstat-collect) recorded on 14 October between 16:32 and 16:50.

```
-rw-r----- 1 syslog adm 21231 Oct 14 16:30 /var/log/auth.log
yateesh@Ubuntu:~/linux-log-monitor$ journalctl -n 10 --no-pager
Oct 14 16:32:03 Ubuntu systemd[1]: anacron.service: Deactivated successfully.
Oct 14 16:36:26 Ubuntu systemd[1]: Starting systemd-tmpfiles-clean.service - Cleanup of Temporary Directories
...
Oct 14 16:36:26 Ubuntu systemd[1]: systemd-tmpfiles-clean.service: Deactivated successfully.
Oct 14 16:36:26 Ubuntu systemd[1]: Finished systemd-tmpfiles-clean.service - Cleanup of Temporary Directories
.
Oct 14 16:40:04 Ubuntu systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
Oct 14 16:40:04 Ubuntu systemd[1]: sysstat-collect.service: Deactivated successfully.
Oct 14 16:40:04 Ubuntu systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
Oct 14 16:50:04 Ubuntu systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
Oct 14 16:50:04 Ubuntu systemd[1]: sysstat-collect.service: Deactivated successfully.
Oct 14 16:50:04 Ubuntu systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
yateesh@Ubuntu:~/linux-log-monitor$ ls -l /var/log/syslog
-rw-r----- 1 syslog adm 1188651 Oct 14 16:50 /var/log/syslog
```

Figure 3.13: Permissions of system logs and recent journal entries

The script `real_log_viewer.sh` provides a menu named REAL LINUX LOG VIEWER with five choices: Authentication log, System log, Recent system journal, Sample log, and Back (Figure 3.14). The screenshot shows the menu waiting for a choice; the output of a selected source is not shown in it.

```
yateesh@Ubuntu:~/linux-log-monitor$ nano scripts/real_log_viewer.sh
yateesh@Ubuntu:~/linux-log-monitor$ chmod +x scripts/real_log_viewer.sh
./scripts/real_log_viewer.sh
====================================
      REAL LINUX LOG VIEWER
====================================
1. Authentication log
2. System log
3. Recent system journal
4. Sample log
5. Back
====================================
Choose a log source (1-5):
```

Figure 3.14: Real Linux log viewer menu

The web dashboard offers a similar read-only view, described in Section 3.11 (Figure 3.17). Access to a file depends on the permissions of the user running the application.

3.10 Python Backend and API

A Python 3 script, `server.py`, acts as a lightweight local HTTP server and API bridge. It serves the dashboard page at `http://127.0.0.1:8000` and receives the requests made by the dashboard's JavaScript. For each request, it connects to the corresponding Bash script or file and returns the result to the page. In this way, Bash performs the core monitoring operations, and Python only connects the scripts to the dashboard.

The dashboard operations that depend on this bridge are starting and stopping monitoring, searching the log, detecting failed logins, analyzing the log, generating the report, viewing alerts, and loading system logs. The Python backend exposes HTTP GET and POST endpoints to connect the web dashboard with the Bash monitoring scripts. GET requests retrieve system status, log entries, alert history, monitor status, and selected Linux system logs. POST requests perform keyword searches, failed-login detection, report generation, log analysis, and monitoring control operations. Table 3.2 summarizes the dashboard operations, their associated Bash components, and the corresponding API endpoints.

Table 3.2: Dashboard operations served by the backend

Dashboard operation	Bash component	Endpoint
Start Monitoring	auto_monitor.sh	POST /api/monitor/start
Stop Monitoring	auto_monitor.sh	POST /api/monitor/stop
Search logs	search_logs.sh	POST /api/search
Detect Failed Logins	failed_login.sh	POST /api/failed-logins
Analyze Logs	analyzer.sh	POST /api/analyze
Generate Report	generate_report.sh	POST /api/report
View Alerts / Alert history	alerts/automatic_alerts.txt	GET /api/alerts
Real Linux System Logs	Selected system log files	GET /api/system-logs?type=syslog, ?type=auth, or ?type=kernel

3.11 Web Dashboard

The dashboard is a single web page titled Linux Log Monitoring System, with the sub-heading "System overview · Real system logs · Log analysis · Security alerts". Figure 3.15 shows its upper part. It displays the monitoring status (STOPPED in this screenshot), the time of the last dashboard refresh (10:30:42 AM), and four counters: total log entries 23, errors 9, critical events 8, and failed logins 4. The Monitoring controls panel has the buttons Start Monitoring, Stop Monitoring, and Refresh Now, and the Search logs panel has a keyword field and a Search button.

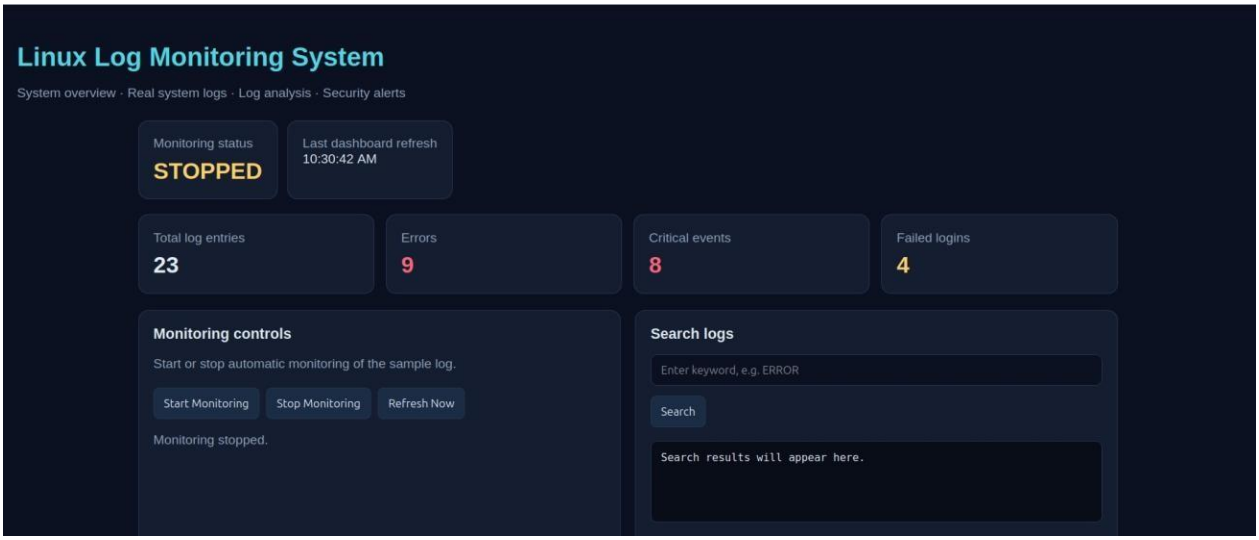


Figure 3.15: Web dashboard overview with status, counters, controls, and search

The Recent log entries panel (Figure 3.16) shows the sample log and refreshes every three seconds. Entries are colour-coded by level: ERROR lines in orange and CRITICAL lines in red. One entry, "ERROR: Test alert from Linux dashboard", has no timestamp and appears to have been added manually as a test.



Figure 3.16: Recent log entries displayed on the dashboard

The Real Linux System Logs panel (Figure 3.17) is described on the page as a read-only view of log files from the Ubuntu system. The user selects a source from a drop-down list and presses Load Logs. In the screenshot, the kernel log `/var/log/kern.log` is selected, and the panel reports that up to 200 recent entries are displayed. Some entries are highlighted in different colours.

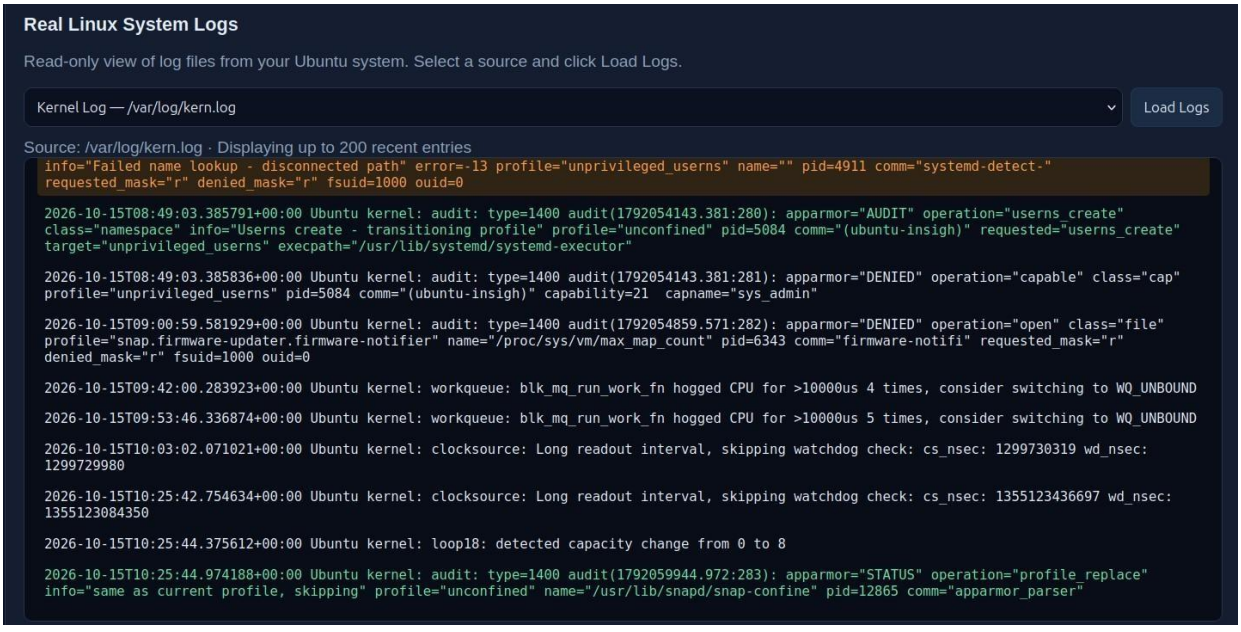


Figure 3.17: Read-only view of the kernel log on the dashboard

The Security and reports panel (Figure 3.18) provides the buttons Detect Failed Logins, Generate Report, Analyze Logs, and View Alerts; their results appear in the output box below them, which in this screenshot shows only “Monitoring stopped.” The Alert history panel refreshes automatically

every five seconds and lists the stored alerts, using the same “[date time] ALERT: …” form as the alert file. The alert for SYNTHETIC_TEST_EVENT_20261015 appears twice in the list, both at 09:35:16.

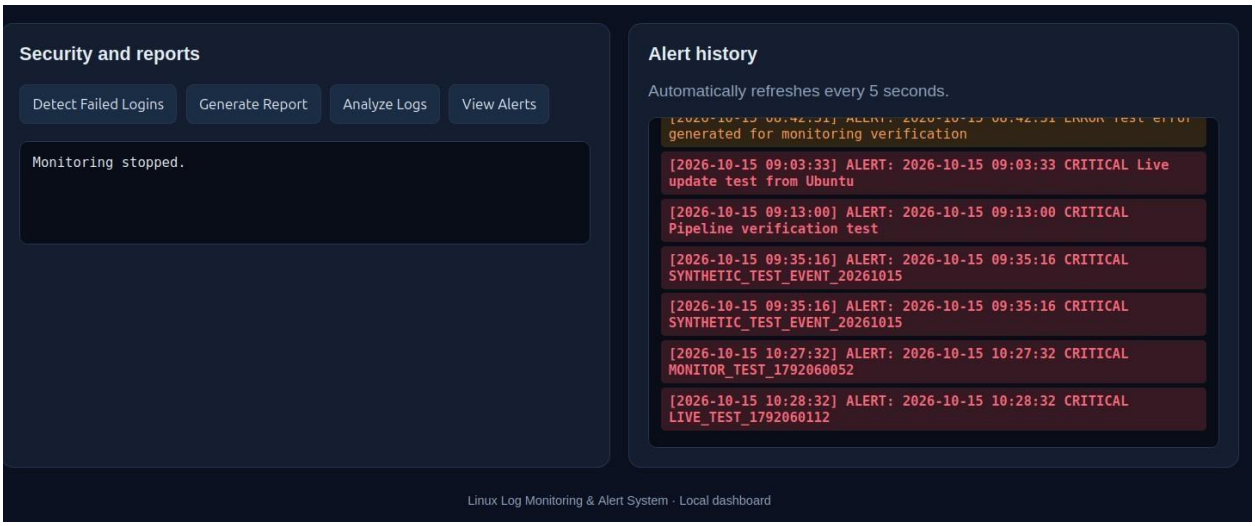


Figure 3.18: Security and reports panel and alert history

3.12 Security Considerations and Limitations

- **Local access only:** the dashboard is addressed at the loopback address 127.0.0.1, so it is meant to be used on the same machine.
- **File permissions:** system logs such as `auth.log` and `syslog` are readable only by their owner and the adm group (Figure 3.13); the viewer can show a file only where the user’s permissions allow it.
- **Read-only viewing:** the dashboard’s system-log view is read-only.
- **No authentication layer:** no login, encryption (HTTPS), or user-role mechanism is described among the implemented features, so the dashboard should not be exposed to a network.
- **Simple keyword matching:** detection depends on keywords and is not case-sensitive, so lines can match unintentionally (Figure 3.9, line 7).
- **Demonstration data:** most functions were tested on a small sample log and on manually added test entries, not on a production workload.

4. Results and Testing

4.1 Implementation Results

The Linux Log Monitoring & Alert System was implemented with Bash scripts, a Python 3 local server, and an HTML, CSS, and JavaScript dashboard on Ubuntu Linux. The results of the individual functions were shown in Chapter 3. This section presents the results of the automatic monitoring tests and of the terminal dashboard, and relates them to the web dashboard.

4.1.1 Automatic Monitoring and Alert Generation

To test the monitor, test entries were appended to `logs/sample.log` from the terminal. In Figure 4.1, `echo` commands append timestamped ERROR entries (“Test alert from monitoring system” and “Test from main menu”), and a `cat >>` command appends three more entries: an ERROR, a CRITICAL, and a FAILED entry. The timestamps of these three entries (18:00:01 to 18:00:03) were typed manually and are later than the machine’s clock at the time of the test.

```
yateesh@Ubuntu:~/linux-log-monitor$ echo "$(date '+%Y-%m-%d %H:%M:%S') ERROR Test alert from monitoring s
m" >> ~/linux-log-monitor/logs/sample.log
yateesh@Ubuntu:~/linux-log-monitor$ echo "$(date '+%Y-%m-%d %H:%M:%S') ERROR Test from main menu" >> ~/li
log-monitor/logs/sample.log
yateesh@Ubuntu:~/linux-log-monitor$ cat >> ~/linux-log-monitor/logs/sample.log <<'EOF'
2026-10-14 18:00:01 ERROR Test application error
2026-10-14 18:00:02 CRITICAL Test critical service failure
2026-10-14 18:00:03 FAILED Test failed login event
EOF
```

Figure 4.1: Test entries appended to the sample log

The monitor was started with `bash -n scripts/auto_monitor.sh && ./scripts/auto_monitor.sh`, which first checks the script syntax and then runs it. Figure 4.2 shows the heading AUTOMATIC LOG MONITOR, the monitored file, and the message “Press Ctrl+C to stop.”, followed by the alert “[2026-10-14 17:18:52] ALERT: 2026-10-14 17:18:52 ERROR Test alert from monitoring system”. The alert carries the same timestamp as the test entry written by the first `echo` command.

```
yateesh@Ubuntu:~/linux-log-monitor$ chmod +x scripts/auto_monitor.sh
bash -n scripts/auto_monitor.sh && ./scripts/auto_monitor.sh
=====================================
     AUTOMATIC LOG MONITOR
=====================================
Monitoring: /home/yateesh/linux-log-monitor/logs/sample.log
Press Ctrl+C to stop.
=====================================
[2026-10-14 17:18:52] ALERT: 2026-10-14 17:18:52 ERROR Test alert from monitoring system
```

Figure 4.2: Automatic log monitor raising an alert

The alert was also stored. Figure 4.3 shows `cat ~/linux-log-monitor/alerts/automatic_alerts.txt` returning the same alert line.

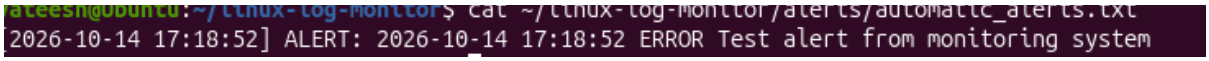


Figure 4.3: Alert stored in automatic_alerts.txt

4.1.2 Monitoring from the Menu

The ten-option menu starts the monitor with option 10. In Figure 4.4, option 10 was chosen and the monitor reported the three entries appended in Figure 4.1 as three alerts: ERROR (Test application error), CRITICAL (Test critical service failure), and FAILED (Test failed login event), all displayed at 17:29:32. The menu in this screenshot has the entries View sample log, Search logs by keyword, Detect failed login attempts, Generate log summary report, View recent alerts, View system information, Run full log analysis, View real Linux logs, Exit, and Start automatic log monitoring.

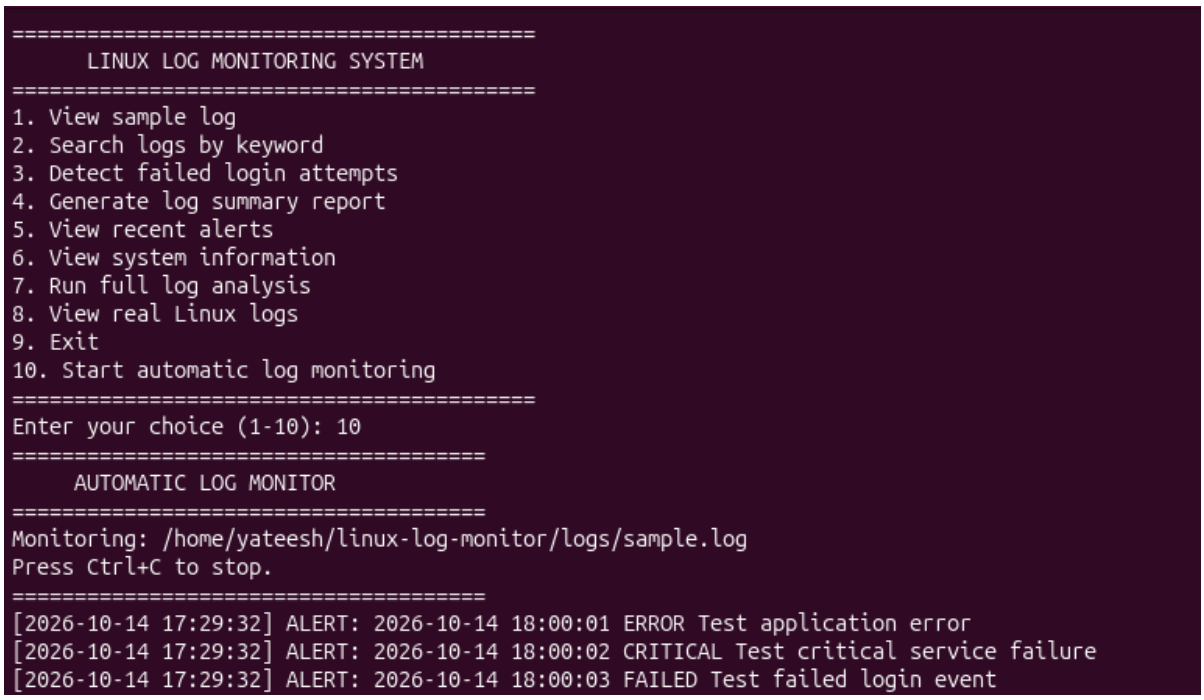


Figure 4.4: Ten-option menu starting the automatic monitor

4.1.3 Terminal Dashboard

The terminal front end also shows a real-time dashboard above the menu (Figure 4.5). It prints the host (Ubuntu), the user (yateesh), the log source, and four counters: total log lines 15, error entries 7, critical events 2, and failed logins 4. These values are larger than those in Figure 3.8 because the sample log had grown as test entries were added.

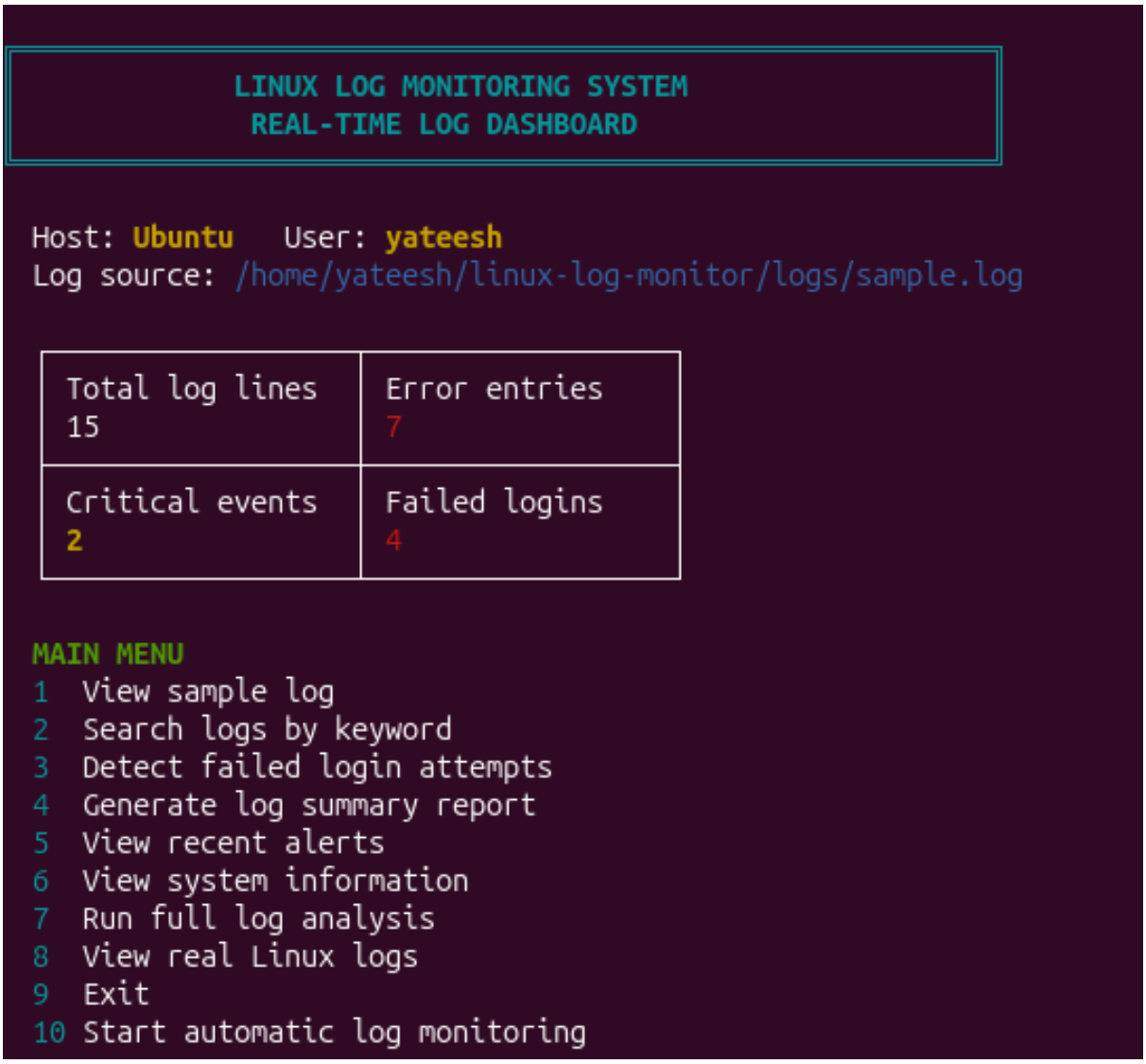


Figure 4.5: Terminal dashboard with counters and main menu

4.1.4 Web Dashboard Results

The web dashboard (Figures 3.15 to 3.18) showed the monitoring status, the four counters, the colour-coded recent log entries, the kernel log view, and the alert history. Its counters (23 entries, 9 errors, 8 critical events, 4 failed logins) differ from those of the terminal dashboard because the log continued to grow between the two sessions.

4.2 Test Environment

Table 4.1: Test environment

Item	Details
Operating system	Ubuntu 26.04.1 LTS (codename resolute)
Virtualisation	VirtualBox virtual machine

Item	Details
User account / host name	yateesh / Ubuntu
Project directory	/home/yateesh/linux-log-monitor
Test data	logs/sample.log (8 original entries plus appended test entries)
Web dashboard address	http://127.0.0.1:8000
Test dates	Terminal tests: 14 October 2026; dashboard screenshots: 15 October 2026

4.3 Testing and Validation

Table 4.2 lists the test cases, the expected result, the result actually observed in the screenshots, and the status. Functions for which no screenshot evidence is available are marked accordingly and are not presented as verified.

Table 4.2: Test cases and results

No.	Test case	Expected result	Actual result (evidence)	Status
1	Project directory creation	Directory and four subfolders exist	scripts, logs, reports, alerts listed (Fig. 3.3, 3.4)	Pass
2	Log analysis	Counts match the sample log	8 total, 2 error, 1 critical, 2 failed login (Fig. 3.8)	Pass
3	Keyword search: ERROR	Lines containing ERROR	Lines 2 and 7 returned (Fig. 3.9)	Pass
4	Keyword search: FAILED	Lines containing FAILED	Lines 3, 4, and 7 returned; line 7 matched on “failed” (Fig. 3.9)	Pass (note)
5	Failed-login detection	Failed-login lines listed, alert saved	Lines 3 and 4 listed; alert saved message shown (Fig. 3.10)	Pass
6	Summary report	Report displayed and saved	Report shown and saved to log_summary.txt; file content matches (Fig. 3.11, 3.12)	Pass
7	Menu: view log (option 1)	All entries displayed	Eight entries displayed (Fig. 3.6)	Pass
8	Menu: invalid input	Error message, return to menu	Message for 99 shown (Fig. 3.7)	Pass
9	System log permissions and journal	Permissions visible; journal output shown	Permissions and ten journal entries shown (Fig. 3.13)	Pass
10	Real log viewer menu	Source output displayed	Menu shown; output of a selection not captured (Fig. 3.14)	Partially verified

No.	Test case	Expected result	Actual result (evidence)	Status
11	Automatic monitor alert	Alert for new ERROR entry	ALERT line displayed (Fig. 4.2)	Pass
12	Alert storage	Alert written to file	Alert found in automatic_alerts.txt (Fig. 4.3)	Pass
13	Monitor from menu (option 10)	Alerts for ERROR, CRITICAL, FAILED entries	Three alerts displayed (Fig. 4.4)	Pass
14	Terminal dashboard	Counters and menu displayed	15 / 7 / 2 / 4 counters shown (Fig. 4.5)	Pass
15	Web dashboard display	Status, counters, recent entries shown	Displayed with live panels (Fig. 3.15, 3.16)	Pass
16	Web system-log view	Selected log loaded read-only	kern.log entries displayed (Fig. 3.17)	Pass
17	Web alert history	Stored alerts listed	Alerts listed (Fig. 3.18)	Pass
18	Start / Stop Monitoring on the dashboard	Status changes	STOPPED status visible; start and stop actions not captured	Pass
19	Dashboard search and security buttons	Results in the output box	No result screenshots available	Pass

4.4 Testing Limitations

- Testing used a small sample log and manually added test entries; no large-scale or performance testing was carried out.
- The timestamps of some appended entries were typed manually, so they do not reflect real system activity.
- Testing was performed on a single Ubuntu virtual machine and a single user account.
- Failed-login detection was tested on sample entries, not on real authentication events.
- The alert for SYNTHETIC_TEST_EVENT_20261015 appears twice in the alert history (Figure 3.18); the cause was not investigated.
- The dashboard's Start/Stop Monitoring, search, and security buttons are not covered by screenshots in this version of the report.

5. Conclusion and Future Scope

5.1 Conclusion

The Linux Log Monitoring & Alert System demonstrates how standard Linux tools can be combined into a practical log monitoring solution. Bash scripts analyse a log, search it by keyword, detect failed-login entries, generate a summary report, view system logs, and watch a log file to raise alerts automatically. A Python 3 local server and an HTML, CSS, and JavaScript dashboard make these functions available in a browser.

The tests showed that the counts, search results, reports, and alerts agreed with the contents of the sample log, and that new entries appended to the log were detected by the automatic monitor. The project gave practical experience in Linux system administration, shell scripting, file permissions, log handling, and the integration of scripts with a simple web interface.

5.2 Achievements

- A modular set of Bash scripts for monitoring, searching, analysis, failed-login detection, report generation, and log viewing.
- Automatic keyword-based alert generation with alerts saved to a text file.
- An interactive terminal menu with input validation, extended from eight to ten options.
- A local web dashboard with status, counters, colour-coded log display, read-only system log view, and alert history.
- A saved text summary report of the log.

5.3 Current Limitations

- Detection is based on simple keywords and is not case-sensitive.
- Monitoring is started and stopped manually; no scheduled execution is configured.
- The dashboard has no authentication and is intended for local use only.
- Alerts are shown on screen and in files only; there are no email or messaging notifications.
- Access to some system logs depends on file permissions.
- Testing was limited to a small sample log and a single virtual machine.

5.4 Future Scope

- Schedule monitoring with Cron or a systemd service.
- Send alerts by email or messaging services.
- Add authentication and HTTPS to the dashboard.
- Monitor additional logs and analyse real authentication logs for repeated failed logins.
- Use pattern rules and severity levels instead of fixed keywords.

- Store log history in a database and add charts and trends.
- Support monitoring of multiple machines.
- Export reports in other formats such as PDF.

References

1. GNU Project, GNU Bash Reference Manual. Available at: https://www.gnu.org/software/bash/manual/bash.html.

2. GNU Project, GNU Coreutils Manual. Available at: https://www.gnu.org/software/coreutils/manual/coreutils.html.

3. Ubuntu Documentation, Ubuntu Server Documentation. Available at: https://ubuntu.com/server/docs.

4. systemd Project, journalctl — Query the systemd journal (manual page). Available at: https://www.freedesktop.org/software/systemd/man/latest/journalctl.html.

5. Linux Manual Pages Project, Linux Manual Pages (man pages). Available at: https://www.kernel.org/doc/man-pages/.

6. Python Software Foundation, Python 3 Documentation. Available at: https://docs.python.org/3/.

7. Mozilla, MDN Web Docs (HTML, CSS, and JavaScript). Available at: https://developer.mozilla.org/.

8. Oracle, Oracle VM VirtualBox User Manual. Available at: https://www.virtualbox.org/manual/.

Appendices

Appendix A: System Architecture

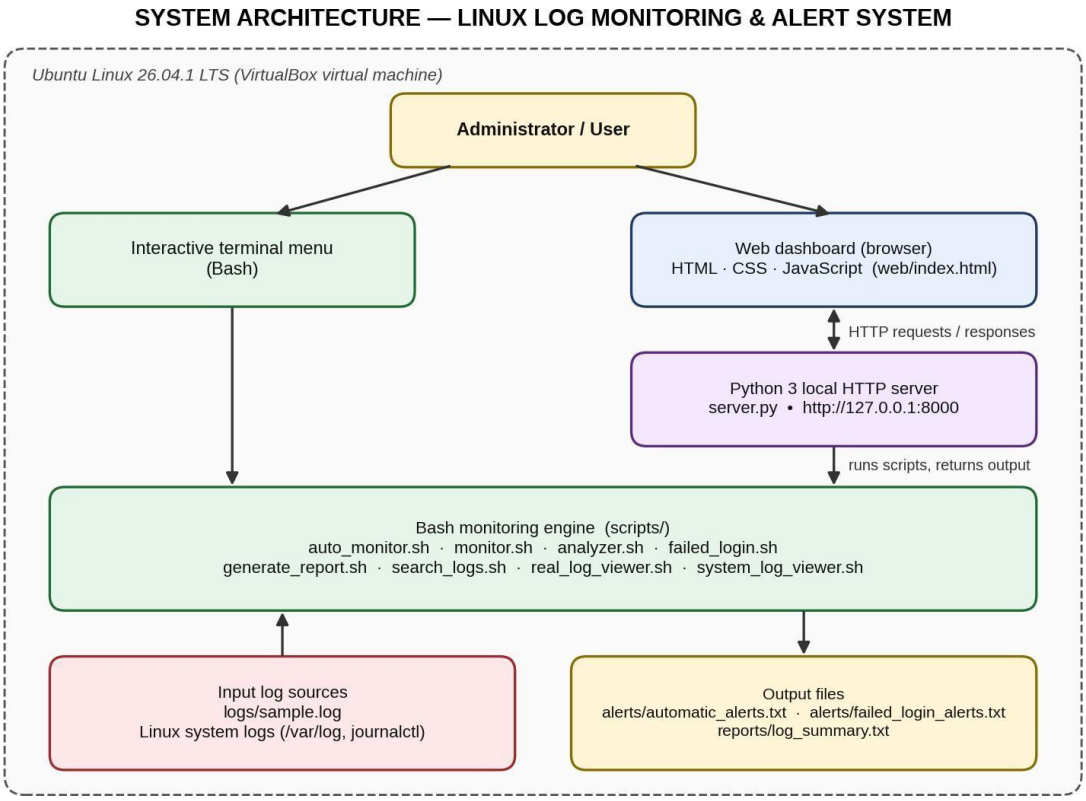


Figure A1: System architecture of the Linux Log Monitoring & Alert System

Appendix B: Linux Commands and Configuration

This appendix lists the main commands used in the project. The corresponding terminal outputs appear in the figures cited.

B.1 System Information Commands (Figure 3.2)

```
whoami
hostname
lsb_release -a
```

B.2 Project Directory Creation and Listing (Figures 3.3, 3.4)

```
mkdir -p ~/linux-log-monitor/{scripts,logs,reports,alerts}
cd ~/linux-log-monitor
pwd
ls -R
find . -maxdepth 3 -type f | sort
```

B.3 Making Scripts Executable and Running Them (Figures 3.6 to 3.12)

```
nano scripts/analyzer.sh
chmod +x scripts/analyzer.sh
./scripts/analyzer.sh
./scripts/search_logs.sh
./scripts/failed_login.sh
./scripts/generate_report.sh
cat reports/log_summary.txt
```

B.4 System Log Commands (Figures 3.13, 3.14)

```
ls -l /var/log/auth.log
ls -l /var/log/syslog
journalctl -n 10 --no-pager
./scripts/real log viewer.sh
```

B.5 Automatic Monitoring Tests (Figures 4.1 to 4.3)

```
chmod +x scripts/auto_monitor.sh
bash -n scripts/auto_monitor.sh && ./scripts/auto_monitor.sh
cat ~/linux-log-monitor/alerts/automatic_alerts.txt
```

B.6 Important Project Files

Table B1: Important project files

File or directory	Purpose
scripts/	Bash monitoring and log-processing scripts
server.py	Python 3 local HTTP server and API bridge
web/index.html	Web dashboard (HTML, CSS, JavaScript)
logs/sample.log	Sample log used for testing
alerts/automatic_alerts.txt	Alerts from the automatic monitor
alerts/failed_login_alerts.txt	Alerts from failed-login detection
reports/log_summary.txt	Generated summary report

Appendix C: Shell Scripts and Source Code

The following source listings describe the principal components of the Linux Log Monitoring & Alert System. The system uses Bash scripts for log processing and monitoring, Python 3 for the local HTTP server and API, and HTML, CSS, and JavaScript for the browser-based dashboard.

C.1 Automatic Log Monitoring — scripts/auto_monitor.sh

Purpose: Monitors the sample log for newly added entries and records alerts when important events are detected.

```
#!/bin/bash

PROJECT_DIR="$HOME/linux-log-monitor"
LOG_FILE="$PROJECT_DIR/logs/sample.log"
ALERT_FILE="$PROJECT_DIR/alerts/automatic_alerts.txt"

mkdir -p "$PROJECT_DIR/alerts"

echo "Linux Log Monitoring - Automatic Monitor"
echo "Press Ctrl+C to stop monitoring."

touch "$ALERT_FILE"

LAST_LINE=$(wc -l < "$LOG_FILE")

while true; do
   CURRENT_LINE=$(wc -l < "$LOG_FILE")

   if [ "$CURRENT_LINE" -gt "$LAST_LINE" ]; then
      NEW_LINES=$((CURRENT_LINE - LAST_LINE))

      tail -n "$NEW_LINES" "$LOG_FILE" |
      while IFS= read -r line; do
```

```
        if echo "$line" | grep -Eiq 'ERROR|CRITICAL|FAILED|failure'; then
            TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
            echo "[$TIMESTAMP] ALERT: $line" |
                tee -a "$ALERT_FILE"
        fi
    done

    LAST_LINE="$CURRENT_LINE"
  fi

  sleep 5
done
```

C.2 Main Menu — scripts/monitor.sh

Purpose: Provides a menu-driven terminal interface for accessing the monitoring system's features.

```
#!/bin/bash

PROJECT_DIR="$HOME/linux-log-monitor"
LOG_FILE="$PROJECT_DIR/logs/sample.log"

while true; do
  clear
  echo "========================================="
  echo " LINUX LOG MONITORING & ALERT SYSTEM"
  echo "========================================="
  echo "1. View Sample Log"
```

```
    echo "2. Search Logs"
    echo "3. Detect Failed Login Attempts"
    echo "4. Generate Report"
    echo "5. View Recent Alerts"
    echo "6. System Information"
    echo "7. Full Log Analysis"
    echo "8. View Real Linux Logs"
    echo "9. Exit"
    echo "10. Start Automatic Monitoring"
    echo "========================================="

    read -rp "Enter your choice: " choice

    case "$choice" in
        1)
            cat "$LOG_FILE"
            ;;
        2)
            "$PROJECT_DIR/scripts/search_logs.sh"
            ;;
        3)
            "$PROJECT_DIR/scripts/failed_login.sh"
            ;;
        4)
            "$PROJECT_DIR/scripts/generate_report.sh"
            ;;
```

```
5)
    cat "$PROJECT_DIR/alerts/failed_login_alerts.txt" 2>/dev/null
    cat "$PROJECT_DIR/alerts/automatic_alerts.txt" 2>/dev/null
    ;;
6)
    hostname
    whoami
    uptime
    df -h /
    ;;
7)
    "$PROJECT_DIR/scripts/analyzer.sh"
    ;;
8)
    "$PROJECT_DIR/scripts/real_log_viewer.sh"
    ;;
9)
    echo "Exiting the monitoring system."
    exit 0
    ;;
10)
    "$PROJECT_DIR/scripts/auto_monitor.sh"
    ;;
*)
    echo "Invalid choice. Please try again."
    ;;
```

```
    esac


    read -rp "Press Enter to continue..."
done
```

C.3 Log Analyzer — scripts/analyzer.sh

Purpose: Analyzes the sample log and counts important log categories.

```
#!/bin/bash

PROJECT_DIR="$HOME/linux-log-monitor"
LOG_FILE="$PROJECT_DIR/logs/sample.log"

if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Log file not found."
    exit 1
fi

echo "========== LOG ANALYSIS =========="
echo "Total log entries: $(wc -l < "$LOG_FILE")"
echo "ERROR entries: $(grep -ci 'ERROR' "$LOG_FILE")"
echo "CRITICAL entries: $(grep -ci 'CRITICAL' "$LOG_FILE")"
echo "Failed login entries: $(grep -Eic 'FAILED|failed login' "$LOG_FILE")"
echo "=================================="
```

C.4 Failed Login Detection — scripts/failed_login.sh

Purpose: Identifies failed-login messages in the sample log and saves the matching entries to an alert file.

```
#!/bin/bash
```

```
PROJECT_DIR="$HOME/linux-log-monitor"
LOG_FILE="$PROJECT_DIR/logs/sample.log"
ALERT_FILE="$PROJECT_DIR/alerts/failed_login_alerts.txt"

mkdir -p "$PROJECT_DIR/alerts"

if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Sample log not found."
    exit 1
fi

echo "Failed Login Detection"
grep -Ein 'failed login|authentication failure|invalid password' \
    "$LOG_FILE" > "$ALERT_FILE"

if [ -s "$ALERT_FILE" ]; then
    echo "Failed login events detected:"
    cat "$ALERT_FILE"
else
    echo "No failed login events found."
fi
```

C.5 Report Generation — scripts/generate_report.sh

Purpose: Creates a text report containing log counts and failed-login information.

```
#!/bin/bash
```

```
PROJECT_DIR="$HOME/linux-log-monitor"
LOG_FILE="$PROJECT_DIR/logs/sample.log"
REPORT_FILE="$PROJECT_DIR/reports/log_summary.txt"

mkdir -p "$PROJECT_DIR/reports"

if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Sample log not found."
    exit 1
fi

{
    echo "LINUX LOG MONITORING REPORT"
    echo "Generated: $(date)"
    echo "----------------------"
    echo "Total entries: $(wc -l < "$LOG_FILE")"
    echo "ERROR entries: $(grep -ci 'ERROR' "$LOG_FILE")"
    echo "CRITICAL entries: $(grep -ci 'CRITICAL' "$LOG_FILE")"
    echo "Failed login entries: $(grep -Eic 'FAILED|failed login' "$LOG_FILE")"
    echo "----------------------"
} > "$REPORT_FILE"

echo "Report generated: $REPORT_FILE"
cat "$REPORT_FILE"
```

C.6 Log Search — scripts/search_logs.sh

Purpose: Searches the sample log for entries containing a user-provided keyword.

```
#!/bin/bash

PROJECT_DIR="$HOME/linux-log-monitor"
LOG_FILE="$PROJECT_DIR/logs/sample.log"

if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Sample log not found."
    exit 1
fi

read -rp "Enter keyword to search: " keyword

if [ -z "$keyword" ]; then
    echo "Error: Search keyword cannot be empty."
    exit 1
fi

grep -inF -- "$keyword" "$LOG_FILE"

if [ "$?" -ne 0 ]; then
    echo "No matching entries found."
Fi
```

C.7 Real Linux Log Viewer — scripts/real_log_viewer.sh

Purpose: Displays selected system log information where the current user has permission to access it.

```
#!/bin/bash


echo "========== REAL LINUX LOG VIEWER =========="
echo "1. Authentication log"
echo "2. System journal"
echo "3. Recent kernel messages"
read -rp "Choose a source: " choice


case "$choice" in
  1)
    if [ -r /var/log/auth.log ]; then
      tail -n 20 /var/log/auth.log
    else
      echo "Authentication log unavailable or permission denied."
    fi
    ;;
  2)
    journalctl -n 20 --no-pager
    ;;
  3)
    journalctl -k -n 20 --no-pager
    ;;
  *)
```

```
        echo "Invalid choice."
        ;;
esac
```

C.8 System Log Viewer — scripts/system_log_viewer.sh

Purpose: Displays a selected system log source.

```
#!/bin/bash

echo "========== SYSTEM LOG VIEWER =========="
echo "1. System log"
echo "2. Authentication log"
echo "3. Kernel log"
read -rp "Choose a log source: " choice

case "$choice" in
    1) FILE="/var/log/syslog" ;;
    2) FILE="/var/log/auth.log" ;;
    3) journalctl -k -n 30 --no-pager; exit 0 ;;
    *)
        echo "Invalid choice."
        exit 1
        ;;
esac

if [ -r "$FILE" ]; then
    tail -n 30 "$FILE"
else
```

```
    echo "Log file unavailable or permission denied: $FILE"
fi
```

C.9 Python Backend — server.py

Purpose: Runs the local HTTP server, serves the dashboard, and provides API endpoints for monitoring operations.

```
#!/usr/bin/env python3

import json
import subprocess
from http.server import BaseHTTPRequestHandler, HTTPServer
from pathlib import Path
from urllib.parse import urlparse, parse_qs

PROJECT_DIR = Path.home() / "linux-log-monitor"
WEB_FILE = PROJECT_DIR / "web" / "index.html"
LOG_FILE = PROJECT_DIR / "logs" / "sample.log"
ALERT_FILE = PROJECT_DIR / "alerts" / "automatic_alerts.txt"

def run_script(name, *args):
    path = PROJECT_DIR / "scripts" / name
    return subprocess.run(
        ["bash", str(path), *args],
        capture_output=True,
        text=True,
        timeout=30
    )
```

```
class Handler(BaseHTTPRequestHandler):
    def send_json(self, data, status=200):
        body = json.dumps(data).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        parsed = urlparse(self.path)
        route = parsed.path

        if route in ("/", "/index.html"):
          if not WEB_FILE.exists():
              self.send_error(404)
              return
           body = WEB_FILE.read_bytes()
           self.send_response(200)
           self.send_header("Content-Type", "text/html; charset=utf-8")
           self.send_header("Content-Length", str(len(body)))
           self.end_headers()
           self.wfile.write(body)

        elif route == "/api/status":
```

```
            lines = LOG_FILE.read_text(errors="replace").splitlines() if LOG_FILE.exists() else []
            self.send_json({
                "total_entries": len(lines),
                "errors": sum("ERROR" in x.upper() for x in lines),
                "critical": sum("CRITICAL" in x.upper() for x in lines),
                "failed_logins": sum("FAILED" in x.upper() for x in lines)
            })

        elif route == "/api/logs":
            lines = LOG_FILE.read_text(errors="replace").splitlines() if LOG_FILE.exists() else []
            self.send_json({"logs": lines[-200:]})

        elif route == "/api/alerts":
            alerts = ALERT_FILE.read_text(errors="replace").splitlines() if ALERT_FILE.exists()
else []
            self.send_json({"alerts": alerts})

        elif route == "/api/monitor/status":
            result = subprocess.run(
                ["pgrep", "-af", "auto_monitor.sh"],
                capture_output=True, text=True
            )
            self.send_json({"running": result.returncode == 0})

        elif route == "/api/system-logs":
            log_type = parse_qs(parsed.query).get("type", ["syslog"])[0]
```

```
        files = {
            "syslog": Path("/var/log/syslog"),
            "auth": Path("/var/log/auth.log")
        }

        if log_type == "kernel":
            result = subprocess.run(
                ["journalctl", "-k", "-n", "50", "--no-pager"],
                capture_output=True, text=True
            )
            self.send_json({"logs": result.stdout.splitlines(), "error": result.stderr})
        elif log_type in files:
            path = files[log_type]
            try:
                self.send_json({"logs": path.read_text(errors="replace").splitlines()[-50:]})
            except OSError as exc:
                self.send_json({"error": str(exc)}, 403)
        else:
            self.send_json({"error": "Invalid log type"}, 400)

    else:
        self.send_error(404)

def do_POST(self):
    parsed = urlparse(self.path)
    route = parsed.path
```

```
        length = int(self.headers.get("Content-Length", "0"))

        try:
            data = json.loads(self.rfile.read(length) or b"{}")
        except json.JSONDecodeError:
            self.send_json({"error": "Invalid JSON"}, 400)
            return

        scripts = {
            "/api/failed-logins": "failed_login.sh",
            "/api/report": "generate_report.sh",
            "/api/analyze": "analyzer.sh"
        }

        if route == "/api/search":
            keyword = str(data.get("keyword", "")).strip()
            if not keyword or len(keyword) > 100:
                self.send_json({"error": "Invalid keyword"}, 400)
                return
            result = subprocess.run(
                ["grep", "-inF", "--", keyword, str(LOG_FILE)],
                capture_output=True, text=True
            )
            self.send_json({"results": result.stdout.splitlines()})

        elif route in scripts:
```

```
            result = run_script(scripts[route])
            self.send_json({
                "output": result.stdout,
                "error": result.stderr,
                "returncode": result.returncode
            })

        elif route == "/api/monitor/start":
            subprocess.Popen(
                ["bash", str(PROJECT_DIR / "scripts" / "auto_monitor.sh")],
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL,
                start_new_session=True
            )
            self.send_json({"message": "Monitor start requested"})

        elif route == "/api/monitor/stop":
            subprocess.run(["pkill", "-f", "auto_monitor.sh"], capture_output=True)
            self.send_json({"message": "Monitor stop requested"})

        else:
            self.send_error(404)

    def log_message(self, format, *args):
        pass
```

```
if __name__ == "__main__":
    server = HTTPServer(("127.0.0.1", 8000), Handler)
    print("Dashboard available at http://127.0.0.1:8000")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nServer stopped.")
        server.server_close()
```

C.10 Web Dashboard — web/index.html

Purpose: Provides a browser-based interface for viewing monitoring statistics, searching logs, reviewing alerts, and controlling monitoring operations.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Linux Log Monitoring & Alert System</title>
<style>
body {
    font-family: Arial, sans-serif;
    background: #101827;
    color: #f3f4f6;
    margin: 0;
    padding: 24px;
}
h1 { color: #60a5fa; }
```

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 16px;
}
.card, section {
  background: #1f2937;
  border-radius: 10px;
  padding: 18px;
  margin-bottom: 18px;
}
button, input, select {
  padding: 10px;
  margin: 5px;
  border-radius: 5px;
  border: 0;
}
button {
  background: #2563eb;
  color: white;
  cursor: pointer;
}
pre {
  white-space: pre-wrap;
  overflow-wrap: anywhere;
  max-height: 350px;
```

```
    overflow-y: auto;
}
.error { color: #f87171; }
.critical { color: #fbbf24; }
</style>
</head>
<body>
<h1>Linux Log Monitoring & Alert System</h1>
<p>Local log monitoring dashboard</p>

<div class="cards">
    <div class="card">Total Entries<h2 id="total">0</h2></div>
    <div class="card">Errors<h2 id="errors">0</h2></div>
    <div class="card">Critical<h2 id="critical">0</h2></div>
    <div class="card">Failed Logins<h2 id="failed">0</h2></div>
</div>

<section>
    <h2>Monitoring Controls</h2>
    <button onclick="post('/api/monitor/start')">Start Monitoring</button>
    <button onclick="post('/api/monitor/stop')">Stop Monitoring</button>
    <button onclick="refresh()">Refresh Dashboard</button>
    <p id="status">Loading status...</p>
</section>

<section>
```

```
    <h2>Search Logs</h2>
    <input id="keyword" placeholder="Enter keyword">
    <button onclick="searchLogs()">Search</button>
    <pre id="searchResults"></pre>
</section>

<section>
    <h2>Log Operations</h2>
    <button onclick="post('/api/failed-logins')">Detect Failed Logins</button>
    <button onclick="post('/api/analyze')">Analyze Logs</button>
    <button onclick="post('/api/report')">Generate Report</button>
    <pre id="operationResults"></pre>
</section>

<section>
    <h2>Recent Log Entries</h2>
    <pre id="logs">Loading logs...</pre>
</section>

<section>
    <h2>System Logs</h2>
    <select id="logType">
        <option value="syslog">System</option>
        <option value="auth">Authentication</option>
        <option value="kernel">Kernel</option>
    </select>
```

```
    <button onclick="loadSystemLogs()">View System Logs</button>
    <pre id="systemLogs"></pre>
</section>

<section>
    <h2>Alert History</h2>
    <button onclick="loadAlerts()">Refresh Alerts</button>
    <pre id="alerts"></pre>
</section>

<script>
async function getJSON(url) {
    const response = await fetch(url);
    return await response.json();
}

async function post(url, data = {}) {
    const response = await fetch(url, {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify(data)
    });
    const result = await response.json();
    document.getElementById("operationResults").textContent =
        JSON.stringify(result, null, 2);
    await refresh();
```

```
}

async function refresh() {
    try {
        const status = await getJSON("/api/status");
        document.getElementById("total").textContent = status.total_entries ?? 0;
        document.getElementById("errors").textContent = status.errors ?? 0;
        document.getElementById("critical").textContent = status.critical ?? 0;
        document.getElementById("failed").textContent = status.failed_logins ?? 0;

        const logs = await getJSON("/api/logs");
        document.getElementById("logs").textContent =
            (logs.logs || []).join("\n");

        const monitor = await getJSON("/api/monitor/status");
        document.getElementById("status").textContent =
            monitor.running ? "Monitoring status: Running" :
                              "Monitoring status: Stopped";
    } catch (error) {
        document.getElementById("status").textContent =
            "Unable to connect to local server: " + error.message;
    }
}

async function searchLogs() {
    await post("/api/search", {
```

```
        keyword: document.getElementById("keyword").value
    });
    const result = await getJSON("/api/logs");
    document.getElementById("searchResults").textContent =
        (result.logs || []).filter(line =>
            line.toLowerCase().includes(
                document.getElementById("keyword").value.toLowerCase()
            )).join("\n") || "No matching entries found.";
}

async function loadAlerts() {
    const result = await getJSON("/api/alerts");
    document.getElementById("alerts").textContent =
        (result.alerts || []).join("\n") || "No saved alerts.";
}
async function loadSystemLogs() {
    const type = document.getElementById("logType").value;
    const result = await getJSON("/api/system-logs?type=" + type);
    document.getElementById("systemLogs").textContent =
        result.error || (result.logs || []).join("\n");
}
refresh();
loadAlerts();
</script>
</body>
</html>
```